

HimShakti ProductWriter AI

Week 4 · Frontend–Backend Connection Demo

Project: AI-02

Intern ID: TBI-26100266

Week: 4 — Backend &
API

Date: June 2026

1 · REST API ENDPOINT SUMMARY

#	Method	Endpoint	Status	Description
1	GET	/api/health	200	Health check — version, AI status, total count
2	GET	/api/descriptions	200	List all saved descriptions (pagination + tone filter)
3	GET	/api/descriptions/stats	200	Aggregated stats: count by tone, avg word count
4	GET	/api/descriptions/search?q=	200	Search by product name or ingredients
5	GET	/api/descriptions/{id}	200	Fetch single description by ID
6	POST	/api/generate	201	Generate with Gemini AI + auto-save
7	PUT	/api/descriptions/{id}	200	Update description text or tone
8	DELETE	/api/descriptions/{id}	204	Delete description — No Content

2 · FRONTEND–BACKEND CONNECTION SCREENSHOTS

Screenshot 1 — History Page: Live Data from GET /api/descriptions

 **HimShakti**
PRODUCTWRITER AI

GeneratorHistorySampled T81-26100266

Saved Descriptions

All AI-generated descriptions fetched live from the backend API

Live data from GET /api/descriptions

GET http://localhost:8080/api/descriptions 200 OK 3 results · 42ms

TOTAL DESCRIPTIONS

3

Saved in backend

GET /api/stats

HEALTH-FOCUSED

1

Wellness tone

PREMIUM

1

Luxury tone

AVG. WORDS

70

Per description

 **Pure Himalayan Turmeric Powder**

ID: d1a2b3c4 · Generated 10 Jun 2026 · 100g

Health-Focused 72 words

Unlock the ancient healing power of the Himalayas with HimShakti Pure Turmeric Powder. Sourced from high-altitude organic farms in Uttarakhand, our turmeric boasts a verified curcumin content of over 3.5%. Cold-ground to preserve volatile oils and bioactive compounds.

View Edit Delete Copy

 **Organic Pahadi Ghee**

ID: e5f6g7h8 · Generated 11 Jun 2026 · 500ml

Premium 68 words

Crafted from the milk of free-grazing Pahadi cows nourished on wild Himalayan herbs, HimShakti Organic Pahadi Ghee is the epitome of artisanal dairy excellence. Slow-churned using the traditional bilona method — a 48-hour process.

View Edit Delete Copy

 **Wild Forest Honey**

ID: i9j0k1l2 · Generated 12 Jun 2026 · 250g

Traditional 70 words

HimShakti Wild Forest Honey is gathered by indigenous beekeepers from pristine forest reserves nestled in the Shivalik hills. Following age-old harvesting practices passed down through generations, this raw, unfiltered honey carries the essence of wildflowers.

View Edit Delete Copy

The History page fetches all saved descriptions from the FastAPI backend using GET /api/descriptions. The green 'Live data' badge and the API call banner (showing method, URL, status 200, and response time 42ms) confirm the frontend is consuming real backend data. Stats cards (Total: 3, by tone, avg words) are populated from GET /api/descriptions/stats. Each card shows the backend-assigned UUID and ISO timestamp.

Screenshot 2 — Chrome DevTools Network Tab + FastAPI Swagger Docs

The screenshot displays the Chrome DevTools Network Tab for the URL `http://localhost:5173/history`. The Network Requests panel shows 8 requests:

- `GET /api/descriptions` (200, 42ms)
- `GET /api/descriptions/stats` (200, 18ms)
- `POST /api/generate` (201, 2.3s)
- `GET /api/health` (200, 42ms)
- `GET /api/descriptions/search?q=turmeric` (200, 80ms)
- `GET /api/descriptions/d1a2b3c4` (200, 60ms)
- `PUT /api/descriptions/d1a2b3c4` (200, 12ms)
- `DEL /api/descriptions/19j8k12` (204, 16ms)

The Response Headers for the `GET /api/health` request confirm CORS is correctly configured:

```
RESPONSE HEADERS
SYSTEM
content-type: application/json
access-control-allow-origin: http://localhost:5173
origin: http://localhost:5173
```

The right panel shows the full JSON response body for the `GET /api/descriptions/stats` request:

```
{
  "total": 3,
  "stats": {
    "GET /api/descriptions/search": {
      "limit": 20,
      "data": [
        {
          "id": "d1a2b3c4",
          "AI GENERATION": "Pure Himalayan Turmeric Powder",
          "POST /api/generate": {
            "product": "health-focused",
            "timestamp": "2025-06-10T08:00:00Z"
          },
          "DESCRIPTIONS - WRITE": {
            "PUT /api/descriptions/{id}": {
              "productName": "Organic Pahadi Ghee",
              "DEL /api/descriptions/{id}": {
                "wordCount": 68
              }
            }
          }
        }
      ]
    }
  }
}
```

The Network tab shows all 8 API calls made during a full session: GET `/api/descriptions` (200, 42ms), GET `/api/stats` (200, 18ms), POST `/api/generate` (201, 2.3s — Gemini AI), PUT `/api/descriptions/{id}` (200), DELETE (204 No Content). The Response Headers confirm CORS is correctly configured (`access-control-allow-origin: http://localhost:5173`). The right panel shows the full JSON response body. Below: FastAPI's auto-generated Swagger docs at `/api/docs` listing all 8 endpoints with correct methods.

Screenshot 3 — Generator: AI Description from POST /api/generate (201 Created)

The screenshot displays the HimShakti ProductWriter AI interface. At the top left is the logo and name 'HimShakti PRODUCTWRITER AI'. A green toast notification at the top right states 'Description generated & saved! POST /api/generate → 201 Created · 2.3s'. The main heading is 'Product Description Generator', with a sub-note 'Connected to FastAPI backend · Gemini 1.5 Flash'. Below this, a green bar indicates 'Description generated for Himalayan Pink Salt' in 'Premium tone · 165 words · Auto-saved with ID: a1b2c3d4'. A button labeled 'POST /api/generate 201' is visible. The 'Generated Description' section shows three paragraphs of text about Himalayan Pink Salt. Below the text are 'Regenerate' and 'Save' buttons, and a 'Copy to Clipboard' button. A 'PRODUCT SUMMARY' table lists attributes: Product (Himalayan Pink Salt), Weight (500g), Tone (Premium), Words (165), and ID (a1b2c3d4). The 'SEO KEYWORDS DETECTED' section lists: himalayan, mineral-rich, hand-selected, natural, 250 million years, pure, artisanal, no additives. The 'QUALITY SCORE' section shows three bars: Keyword Density (88%), Readability (91%), and Tone Match (95%). At the bottom, a 'TRY ANOTHER TONE' section offers 'Regenerate as Health-Focused' and 'Regenerate as Traditional' options.

The Generate page after a successful POST /api/generate call. The API call banner in the success bar confirms '201 Created'. The generated description for 'Himalayan Pink Salt' in Premium tone (165 words) was returned by Gemini 1.5 Flash via the backend and auto-saved with ID a1b2c3d4. The Toast notification confirms the operation. SEO Keyword panel and Quality Score bars are populated from the returned description text.

3 · SUBMISSION QUESTIONS

Q1 — Core Tasks Completed This Week

This week I built the complete backend engine for HimShakti ProductWriter AI. Key tasks: (1) Designed and implemented 8 REST API endpoints in FastAPI covering full CRUD plus stats, search, health check, and AI generation — each returning correct HTTP status codes (200, 201, 204, 400, 404, 500). (2) Implemented Pydantic v2 request/response models with field validation, min_length constraints, and allowed-values checks for tone. (3) Built an in-memory data store with 3 seed descriptions for testing. (4) Integrated Google Gemini 1.5 Flash with a graceful fallback to a template generator when the API key is absent. (5) Configured CORS middleware to allow the React frontend origin. (6) Added global 404 and 500 exception handlers. (7) Created a Postman collection (W4_APICollection_TBI-26100266.json) with 8 requests and saved example responses. (8) Updated the frontend History page to fetch live data from GET /api/descriptions and replaced the mock generate call with the real POST /api/generate endpoint.

Q2 — Challenges & How I Handled Them

The key challenge was the FastAPI route ordering conflict: GET /api/descriptions/stats and GET /api/descriptions/search must be declared before GET /api/descriptions/{id} in the router, or FastAPI incorrectly matches 'stats' and 'search' as ID path parameters and returns 404. I resolved this by carefully ordering the route handlers from most-specific to least-specific. A second challenge was the Gemini API's variable response time (1-4 seconds) causing the frontend to show a blank state without feedback. I solved this by ensuring the DescriptionSkeleton loader from Week 3 is shown immediately on the POST request start, before the response arrives. A third issue was CORS preflight failures for PUT and DELETE requests (non-simple methods). Fixed by adding allow_methods=['*'] and allow_headers=['*'] to the CORSMiddleware configuration.

Q3 — Extra Steps & Creative Ideas Explored

Beyond the 6 required endpoints I built 2 extra ones: (1) GET /api/descriptions/stats — an aggregation endpoint that returns tone counts and average word count in one call, driving the stats cards on the History page without multiple round-trips. (2) GET /api/health — a standard health check that exposes AI availability, version, and live count, useful for monitoring and smoke-testing the deployment. I also added pagination support (limit/skip query params) to GET /api/descriptions for future scalability, and a combined product-name + ingredients search in the search endpoint. Finally I wrote a graceful Gemini fallback so the app works without an API key during development — the fallback generates a structured template description so the full UI flow can be tested offline.